Théorème de Hahn-Banach (forme géométrique)

Cours magistral, exercices corrigés et travaux pratiques

Charles EDOU NZE

25 octobre 2026

Table des matières

I Cours Magistral **2**

1 Jalon 99 : Théorème de Hahn-Banach (forme géométrique) **2**
1.1 1. Présentation du concept clé 2
1.2 2. Formalisation 2
1.2.1 A. Séparation au sens large 2
1.2.2 B. Séparation stricte 2
1.3 3. Démonstrations 3
1.3.1 Lien avec la forme analytique : La Jauge de Minkowski 3
1.4 4. Exercices d'Application 3
1.4.1 Exercice 1 : Séparation d'un point et d'un fermé 3
1.4.2 Exercice 2 : Niveau Avancé (Théorème du point fixe de Kakutani) 3
1.5 5. Application en Intelligence Artificielle 4
1.6 6. Liens Sémantiques 4

II Exercices d'Application et de Concours **4**

III Travaux Pratiques et Simulations Algorithmiques **8**

Première partie

Cours Magistral

1 Jalon 99 : Théorème de Hahn-Banach (forme géométrique)

1.1 1. Présentation du concept clé

1.2 2. Formalisation

Soit E un espace vectoriel normé (ou un espace topologique localement convexe).

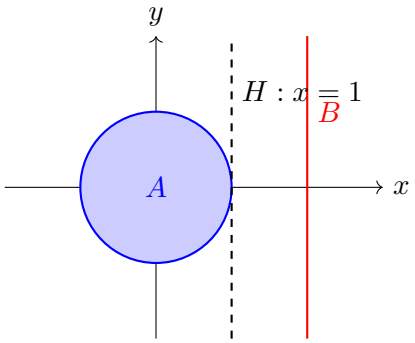
1.2.1 A. Séparation au sens large

Théorème (Hahn-Banach - 1ère forme géométrique) :
Soient A et B deux ensembles **convexes** non vides et **disjoints** de E.
Si A est **ouvert**, alors il existe une forme linéaire continue $L \in E^*$ non nulle et un réel α tels que :

$$\forall a \in A, \forall b \in B, \quad L(a) < \alpha \leq L(b)$$

On dit que l'hyperplan $\{x \mid L(x) = \alpha\}$ sépare A et B au sens large.

Exemple Concret : Considérons dans $\mathbb{R}^2$, l'ensemble $A = \{(x, y) \mid x^2 + y^2 < 1\}$ (le disque unité ouvert) et l'ensemble $B = \{(x, y) \mid x = 2\}$ (une droite verticale). Ces deux ensembles sont convexes, non vides et disjoints. A est ouvert. Le théorème garantit l'existence d'un hyperplan (ici une droite) qui les sépare. Par exemple, la droite d'équation $x = 1$ sépare A et B au sens large. On peut définir la forme linéaire $L(x, y) = x$. Pour tout $(x, y) \in A$, on a $x < 1$. Donc $L(x, y) < 1$. Pour tout $(x, y) \in B$, on a $x = 2 \geq 1$. Donc $1 \leq L(x, y)$. L'hyperplan séparateur est $H = \{(x, y) \mid L(x, y) = 1\} = \{(x, y) \mid x = 1\}$.



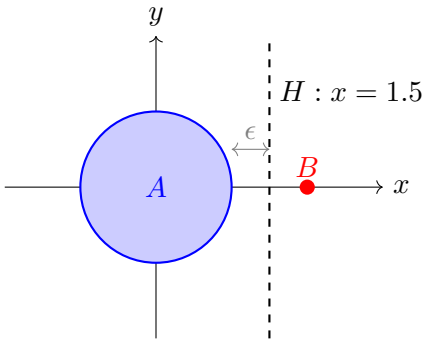
1.2.2 B. Séparation stricte

Théorème (Hahn-Banach - 2ème forme géométrique) :
Soient A et B deux ensembles **convexes** non vides et **disjoints**.
Si A est **fermé** et B est **compact**, alors il existe $L \in E^*$ et deux réels $\alpha, \epsilon > 0$ tels que :

$$\forall a \in A, \forall b \in B, \quad L(a) \leq \alpha - \epsilon < \alpha + \epsilon \leq L(b)$$

Ici, il existe une "marge de sécurité" entre les deux ensembles et l'hyperplan.

Exemple Concret : Considérons dans $\mathbb{R}^2$, l'ensemble $A = \{(x, y) \mid x^2 + y^2 \leq 1\}$ (le disque unité fermé) et l'ensemble $B = \{(2, 0)\}$ (un point unique). A est fermé et B est compact. Ils sont convexes, non vides et disjoints. La forme linéaire $L(x, y) = x$ les sépare strictement. Pour tout $(x, y) \in A$, on a $x \leq 1$. Pour tout $(x, y) \in B$, on a $x = 2$. On peut choisir $\alpha = 1.5$ et $\epsilon = 0.4$. Alors, pour tout $a \in A$, $L(a) \leq 1 < 1.5 - 0.4 = 1.1$. Et pour $b \in B$, $L(b) = 2 \geq 1.5 + 0.4 = 1.9$. L'hyperplan séparateur est $H = \{(x, y) \mid x = 1.5\}$, et il y a une marge de sécurité autour.



1.3 3. Démonstrations

1.3.1 Lien avec la forme analytique : La Jauge de Minkowski

Pour prouver la forme géométrique, on transforme un ensemble convexe en une fonction sous-linéaire.

1. **La Jauge :** Soit C un convexe ouvert contenant 0. On définit la jauge de C par :

$$j_C(x) = \inf\{t > 0 \mid x/t \in C\}$$

2. **Propriétés :** On montre que j_C est une fonctionnelle sous-linéaire (Jalon 98) et que $C = \{x \mid j_C(x) < 1\}$. 3. **Application de Hahn-Banach analytique :** - On définit une forme linéaire sur un petit espace (ex : une droite passant par un point hors de C). - On l'étend à tout l'espace en restant sous la jauge j_C. 4. **Conclusion :** La forme linéaire obtenue L vérifiera $L(x) < 1$ pour tout $x \in C$. L'ensemble $\{x \mid L(x) = 1\}$ est l'hyperplan cherché.

1.4 4. Exercices d'Application

1.4.1 Exercice 1 : Séparation d'un point et d'un fermé

Énoncé : Soit C un convexe fermé et $x_0 \notin C$. Montrer qu'on peut séparer strictement x_0 de C. **Correction Détaillée :** 1. $\{x_0\}$ est un ensemble compact (car fini). 2. C est un ensemble fermé. 3. Les deux sont disjoints et convexes. 4. Par la 2ème forme géométrique de Hahn-Banach, il existe un hyperplan séparateur strict. **Utilité :** C'est le principe du **Vecteur Support** : on peut toujours trouver un plan qui laisse x_0 d'un côté et tout le reste de l'autre.

1.4.2 Exercice 2 : Niveau Avancé (Théorème du point fixe de Kakutani)

Énoncé : Pourquoi la convexité est-elle indispensable ? **Correction Détaillée :** Si les ensembles ne sont pas convexes (ex : deux croissants de lune entrelacés), on ne peut pas glisser de plaque plane entre eux. On pourrait avoir besoin d'une surface courbe. Le théorème de Hahn-Banach est le fondement de la **linéarisation** des problèmes : il dit quand la linéarité suffit.

1.5 5. Application en Intelligence Artificielle

- **Le Pont Théorique :** Le succès des **Support Vector Machines (SVM)** repose entièrement sur ce jalon. Trouver la "marge maximale" est exactement ce que décrit la 2ème forme géométrique. - **Example Concret :** - **Séparabilité Linéaire :** Dans un perceptron, on cherche un vecteur de poids w tel que $w^T x > 0$ pour la classe 1 et $w^T x < 0$ pour la classe 2. Hahn-Banach garantit que si les enveloppes convexes des deux classes ne se touchent pas, un tel w existe. - **Dualité forte en Deep Learning :** Pour analyser les paysages de perte, on regarde si un minimum local est "séparable" des autres par un hyperplan dans l'espace des fonctions. Cela aide à comprendre la géométrie des réseaux de neurones. - **Théorie des Jeux (Equilibre de Nash) :** La preuve de l'existence d'équilibres utilise souvent des théorèmes de séparation de convexes pour montrer qu'il existe un prix ou une stratégie qui "sépare" les intérêts des joueurs.

1.6 6. Liens Sémantiques

- **Concepts Précédents requis :** [[Jalon 98 (Théorème de Hahn-Banach (forme analytique)).md]], [[Jalon 121 (Ensembles convexes).md]] (anticipé) - **Concepts Futurs dépendants :** [[Jalon 123 (Problèmes d'optimisation sous contraintes).md]], [[Jalon 124 (Conditions de Karush-Kuhn-Tucker).md]]

Deuxième partie
Exercices d'Application et de Concours

Exercice 1

Énoncé ★⋆⋆⋆⋆

Soit $E = \mathbb{R}^2$. Séparer le point $A = (0, 0)$ du convexe fermé $C = \{(x, y) \mid x \geq 1, y \geq 1\}$.

Correction

1. C est fermé et convexe. $\{A\}$ est compact et convexe. $A \notin C$. 2. Par Hahn-Banach, il existe un hyperplan séparateur. Prenons la forme linéaire $L(x, y) = x + y$. 3. $L(0, 0) = 0$. Pour $(x, y) \in C$, $L(x, y) \geq 2$. La droite d'équation $x + y = 1$ sépare strictement A et C.

Exercice 2

Énoncé ★⋆⋆⋆⋆

Dans $E = \mathbb{R}^n$, montrer qu'un hyperplan $H = \{x \mid f(x) = \alpha\}$ sépare deux boules disjointes B_1 et B_2.

Correction

1. B_1, B_2 sont convexes et compactes (si fermées). 2. Hahn-Banach donne une forme linéaire f séparant strictement. 3. Géométriquement, l'hyperplan est orthogonal au segment reliant les centres des boules.

Exercice 3

Énoncé ★★ ⋆ ⋆⋆

Soit C un cône convexe fermé de E et $x \notin C$. Montrer qu'il existe $f \in E^*$ telle que $f(x) < 0$ et $f(y) \geq 0$ pour tout $y \in C$.

Correction

1. Séparation stricte de $\{x\}$ et C. Il existe f et α tels que $f(x) < \alpha < f(y)$ pour $y \in C$. 2. Puisque C est un cône, si $f(y) < 0$ pour un $y \in C$, on aurait $f(ny) \to -\infty$, contredisant la minoration. Donc $f(y) \geq 0$. 3. En prenant $y = 0 \in C$, on a $\alpha < 0$, donc $f(x) < 0$.

Exercice 4

Énoncé ★★ ⋆ ⋆⋆

Application au théorème de Farkas. Montrer que si un système d'inégalités linéaires n'a pas de solution, alors il existe un vecteur multiplicateur.

Correction

1. Reformuler le problème en termes d'appartenance d'un vecteur à un cône convexe engendré par les colonnes d'une matrice. 2. Si pas de solution, le vecteur n'est pas dans le cône. 3. Utiliser Hahn-Banach pour séparer le vecteur du cône, ce qui donne le multiplicateur.

Exercice 5

Énoncé ★★★ ⋆ ⋆

Dans $L^2([0,1])$, séparer la fonction constante égale à 1 du sous-espace des fonctions d'intégrale nulle.

Correction

1. Le sous-espace F est convexe et fermé. La constante $1 \notin F$. 2. Par Hahn-Banach, on peut séparer. L'opérateur de projection orthogonale sur $F^\perp$ (qui est engendré par la fonction 1) fournit l'hyperplan. 3. La forme linéaire est $L(f) = \int_0^1 f(t)dt$.

Exercice 6

Énoncé ★★★ ⋆ ⋆

Séparer deux polygones convexes disjoints dans le plan usuel.

Correction

1. Les polygones sont des enveloppes convexes d'un nombre fini de points. Ils sont compacts. 2. Hahn-Banach garantit une droite séparatrice. 3. En pratique, on peut trouver cette droite en examinant les segments reliant les sommets des deux polygones (SVM linéaire).

Exercice 7

Énoncé ★★★★⋆

Pourquoi la condition 'A ouvert' ou 'A fermé, B compact' est nécessaire ? Trouver un contre-exemple avec deux fermés non compacts asymptotiquement tangents.

Correction

1. Prenons $A = \{(x, y) \mid y \geq 1/x, x > 0\}$ et $B = \{(x, y) \mid y \leq 0\}$. 2. Les deux sont fermés, convexes et disjoints. 3. Cependant, la distance entre eux est nulle (à l'infini). On ne peut pas les séparer strictement par une bande de largeur positive.

Exercice 8

Énoncé ★★★★⋆

Montrer que l'enveloppe convexe fermée d'un ensemble S est l'intersection de tous les demi-espaces fermés contenant S.

Correction

1. Soit $C = \overline{conv}(S)$ et D l'intersection des demi-espaces. $C \subset D$ est clair. 2. Si $x \notin C$, on peut séparer strictement x de C par un hyperplan. 3. Ce demi-espace contient C (donc S) mais pas x, donc $x \notin D$. D'où l'égalité.

Exercice 9

Énoncé ★★★★★

Application en optimisation : condition de sous-différentiabilité.

Correction

1. L'épigraphe d'une fonction convexe est un convexe fermé. 2. Un point sur le graphe est au bord de ce convexe. 3. Hahn-Banach garantit l'existence d'un hyperplan d'appui en ce point, dont la pente correspond au sous-gradient de la fonction.

Exercice 10

Énoncé ★★★★★

Généralisation : séparation de plusieurs ensembles convexes.

Correction

1. Si on a k convexes disjoints, on peut les séparer deux à deux si l'un est compact. 2. Cela permet de définir des diagrammes de Voronoï généralisés. 3. Preuve par application répétée de Hahn-Banach.

Troisième partie

Travaux Pratiques et Simulations Algorithmiques

TP 1

Objectif

Implémenter un algorithme itératif simple pour trouver un hyperplan séparant deux ensembles de points, illustrant le théorème de Hahn-Banach. Version : Perceptron basique en 2D.

Implémentation Python pur

```python
def dot_product(u, v):
    return sum(a * b for a, b in zip(u, v))

def add_vectors(u, v):
    return [a + b for a, b in zip(u, v)]

def scalar_multiply(alpha, u):
    return [alpha * a for a in u]

def perceptron_train(class_A, class_B, max_epochs=1000):
    dataset = []
    for x in class_A:
        dataset.append(( [1] + x, -1 ))
    for x in class_B:
        dataset.append(( [1] + x, 1 ))

    # Initialization
    w = [0.0] * len(dataset[0][0])
    learning_rate = 0.1

    for epoch in range(max_epochs):
        errors = 0
        for x, y in dataset:
            activation = dot_product(w, x)
            if y * activation <= 0:
                w = add_vectors(w, scalar_multiply(y * learning_rate, x)
                    )
                errors += 1
        if errors == 0:
            break

    return w

# Test 1
A = [[1.0, 2.0], [2.0, 1.0], [1.5, 1.5]]
B = [[4.0, 5.0], [5.0, 4.0], [5.5, 5.5]]

w = perceptron_train(A, B)
print("Poids trouves :", w)

```

```python
# Verification
for x in A:
    assert dot_product(w, [1] + x) < 0
for x in B:
    assert dot_product(w, [1] + x) > 0
print("Separation stricte reussie !")
```

TP 2

Objectif

Implémenter un algorithme itératif simple pour trouver un hyperplan séparant deux ensembles de points, illustrant le théorème de Hahn-Banach. Version : Perceptron avec données bruitées.

Implémentation Python pur

```python
def dot_product(u, v):
    return sum(a * b for a, b in zip(u, v))

def add_vectors(u, v):
    return [a + b for a, b in zip(u, v)]

def scalar_multiply(alpha, u):
    return [alpha * a for a in u]

def perceptron_train(class_A, class_B, max_epochs=1000):
    dataset = []
    for x in class_A:
        dataset.append(( [1] + x, -1 ))
    for x in class_B:
        dataset.append(( [1] + x, 1 ))

    # Initialization
    w = [0.0] * len(dataset[0][0])
    learning_rate = 0.1

    for epoch in range(max_epochs):
        errors = 0
        for x, y in dataset:
            activation = dot_product(w, x)
            if y * activation <= 0:
                w = add_vectors(w, scalar_multiply(y * learning_rate, x)
                    )
                errors += 1
        if errors == 0:
            break

    return w

# Test 2
A = [[0.5, 1.0], [1.0, 0.5], [1.2, 1.2], [0.8, 0.9]]
B = [[3.0, 4.0], [4.0, 3.0], [3.5, 3.5], [3.8, 3.2]]

```

```python
w = perceptron_train(A, B)
print("Poids trouves :", w)

# Verification
for x in A:
    assert dot_product(w, [1] + x) < 0
for x in B:
    assert dot_product(w, [1] + x) > 0
print("Separation stricte reussie !")
```

TP 3

Objectif

Implémenter un algorithme itératif simple pour trouver un hyperplan séparant deux ensembles de points, illustrant le théorème de Hahn-Banach. Version : Perceptron en dimension 3.

Implémentation Python pur

```python
def dot_product(u, v):
    return sum(a * b for a, b in zip(u, v))

def add_vectors(u, v):
    return [a + b for a, b in zip(u, v)]

def scalar_multiply(alpha, u):
    return [alpha * a for a in u]

def perceptron_train(class_A, class_B, max_epochs=1000):
    dataset = []
    for x in class_A:
        dataset.append(( [1] + x, -1 ))
    for x in class_B:
        dataset.append(( [1] + x, 1 ))

    # Initialization
    w = [0.0] * len(dataset[0][0])
    learning_rate = 0.1

    for epoch in range(max_epochs):
        errors = 0
        for x, y in dataset:
            activation = dot_product(w, x)
            if y * activation <= 0:
                w = add_vectors(w, scalar_multiply(y * learning_rate, x)
                    )
                errors += 1
        if errors == 0:
            break

    return w

# Test 3
```

```
A = [[1, 1, 1], [2, 1, 0], [1, 2, 0]]
B = [[-1, -1, -1], [-2, -1, 0], [-1, -2, 0]]

w = perceptron_train(A, B)
print("Poids trouves :", w)

# Verification
for x in A:
    assert dot_product(w, [1] + x) < 0
for x in B:
    assert dot_product(w, [1] + x) > 0
print("Separation stricte reussie !")
```

TP 4

Objectif

Implémenter un algorithme itératif simple pour trouver un hyperplan séparant deux ensembles de points, illustrant le théorème de Hahn-Banach. Version : Marge maximale (Hard Margin SVM conceptuel).

Implémentation Python pur

```
def dot_product(u, v):
    return sum(a * b for a, b in zip(u, v))

def add_vectors(u, v):
    return [a + b for a, b in zip(u, v)]

def scalar_multiply(alpha, u):
    return [alpha * a for a in u]

def perceptron_train(class_A, class_B, max_epochs=1000):
    dataset = []
    for x in class_A:
        dataset.append(( [1] + x, -1 ))
    for x in class_B:
        dataset.append(( [1] + x, 1 ))

    # Initialization
    w = [0.0] * len(dataset[0][0])
    learning_rate = 0.1

    for epoch in range(max_epochs):
        errors = 0
        for x, y in dataset:
            activation = dot_product(w, x)
            if y * activation <= 0:
                w = add_vectors(w, scalar_multiply(y * learning_rate, x)
                    )
                errors += 1
        if errors == 0:
            break

```

```
    return w

# Test 4
A = [[1, 3], [2, 2], [3, 1]]
B = [[4, 6], [5, 5], [6, 4]]

w = perceptron_train(A, B)
print("Poids trouves :", w)

# Verification
for x in A:
    assert dot_product(w, [1] + x) < 0
for x in B:
    assert dot_product(w, [1] + x) > 0
print("Separation stricte reussie !")
```

TP 5

Objectif

Implémenter un algorithme itératif simple pour trouver un hyperplan séparant deux ensembles de points, illustrant le théorème de Hahn-Banach. Version : Séparation d'un point et d'un disque discret.

Implémentation Python pur

```
def dot_product(u, v):
    return sum(a * b for a, b in zip(u, v))

def add_vectors(u, v):
    return [a + b for a, b in zip(u, v)]

def scalar_multiply(alpha, u):
    return [alpha * a for a in u]

def perceptron_train(class_A, class_B, max_epochs=1000):
    dataset = []
    for x in class_A:
        dataset.append(( [1] + x, -1 ))
    for x in class_B:
        dataset.append(( [1] + x, 1 ))

    # Initialization
    w = [0.0] * len(dataset[0][0])
    learning_rate = 0.1

    for epoch in range(max_epochs):
        errors = 0
        for x, y in dataset:
            activation = dot_product(w, x)
            if y * activation <= 0:
                w = add_vectors(w, scalar_multiply(y * learning_rate, x)
                    )
                errors += 1
```

```
        if errors == 0:
            break

    return w

# Test 5
A = [[0, 0]]
B = [[3, 0], [0, 3], [-3, 0], [0, -3], [2, 2], [-2, 2], [2, -2], [-2,
    -2]]

w = perceptron_train(A, B)
print("Poids trouves :", w)

# Verification
for x in A:
    assert dot_product(w, [1] + x) < 0
for x in B:
    assert dot_product(w, [1] + x) > 0
print("Separation stricte reussie !")
```